Multistrategy Improved Particle Swarm Optimization Algorithm for Path Planning of UAV in 3-D Low Altitude Urban Environment

Fazhan Tao, *Member, IEEE*, Zezheng Chen, Zhikai Wang, Longlong Zhu, and Jun Wang

***Abstract*—The Internet of Things (IoT) system and path planning algorithm provide a technological foundation for autonomous navigation of uncrewed aerial vehicles (UAVs). Geospatial data from the IoT system is transmitted to UAVs through lightweight protocols, and UAVs make optimal path decisions based on these data through optimization algorithms. The combination of the IoT, UAV, and path planning technology constitutes a UAV delivery system, which offers an efficient and economical solution for last-mile logistics in smart cities. Among these, rapid and accurate optimal path planning is crucial for the autonomous delivery of UAVs. Therefore, this article proposes a multistrategy improved particle swarm optimization (PSO) algorithm called MSIPSO. First, the algorithm incorporates a local deadlock jump strategy to increase the success rate of path planning in dense obstacle environments. Second, to mitigate the influence of parameter selection on the algorithm's performance, adaptive nonlinear inertia weights and learning factors are introduced to improve the algorithm's stability. Finally, multiple population differentiation evolution strategies are designed, with different position update equations tailored for populations of varying qualities, which enhances the search efficiency of the algorithm. The simulation results show that MSIPSO outperforms PSO, gray wolf optimizer (GWO), whale optimizer (WOA), elite archive-driven PSO (EAPSO) algorithm, and hybrid GWO and differential evolution (HGWODE) algorithm in terms of convergence speed, accuracy, and stability.**

***Index Terms*—Multistrategy improvement, particle swarm optimization (PSO), path planning, uncrewed aerial vehicle (UAV).**

I. Introduction

As a crucial component of smart cities, logistics systems aim to enhance the efficiency of goods transportation and improve the quality of life for urban residents. However, due to the population siphon effect of cities and the rapid development of global e-commerce, urban logistics transportation systems are facing enormous pressure. A uncrewed aerial vehicles (UAVs) terminal delivery system that integrates Internet of Things (IoT) technology, UAVS, and intelligent algorithms is considered an effective solution to this problem. IoT sensors deployed in smart city infrastructure collect data on environmental factors, cargo status, and position information for UAVs [1], [2]. The UAVs, equipped with high-precision sensors, such as inertial measurement units (IMUs), GPS, and light detection and ranging (LiDAR), interact with the IoT system through lightweight protocols, such as MAVLink and PRLAP-IoD. This process not only enriches the drones' own data systems, but also contributes valuable information back to the IoT system [3]. Intelligent algorithms leverage data from both ends to make decisions, plan optimal routes for UAV delivery, reduce human errors, and improve transportation efficiency. Among these components, exploring efficient path planning algorithms for dense urban environments, as the final step in the system, is particularly beneficial for improving the overall system performance [4].

The autonomous path planning of UAV can be expressed as an optimization problem. It refers to the process of finding the maximum or minimum value of an objective function under given constraints [5], [6]. By integrating assessment criteria like path length, flight altitude, pitch angle, and horizontal angle into the objective function, the quality of the planned path can be accurately evaluated. This enables the UAV to approach the target point at a shorter path cost while avoiding obstacles, completing logistics and distribution tasks [7]. It is an NP-hard problem [8]. These problems, characterized by discontinuity, nonlinearity, multimodality, and indivisibility, cannot be effectively addressed using traditional mathematical programming methods.

Over the past few years, optimization techniques like particle swarm optimization (PSO), whale optimization algorithm (WOA), gray wolf optimizer (GWO), and genetic algorithms (GA) have gained significant popularity for addressing a broad spectrum of optimization challenges [9]. This type of algorithm is constructed based on natural phenomena, such as animal behavior or certain physical phenomena. First, it has the characteristic of simplicity. Second, these heuristic algorithms have strong flexibility [10]. They adopt a nondeductive approach, starting from a random solution and using the objective function to determine the direction of optimization [11]. Ultimately, these algorithms are capable of thoroughly exploring the solution space, and with a sufficient number of iterations, they are consistently able to identify viable solutions. Nevertheless,

Received 4 March 2025; revised 9 June 2025; accepted 11 July 2025. Date of publication 16 July 2025; date of current version 25 September 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62301212 and Grant 62371182; in part by the Henan Province Key Scientific and Technological Projects under Grant 242102221025 and Grant 42102241063; in part by the Key Scientific Research Projects of Universities in Henan Province under Grant 24B590001 and Grant 25A413001; and in part by the Program for Science and Technology Innovation Talents in the University of Henan Province under Grant 23HASTIT021. *(Corresponding author: Zhikai Wang.)*

The authors are with the College of Information Engineering, Henan University of Science and Technology, Luoyang 471000, China (e-mail: zhikaiwang@haust.edu.cn).

Digital Object Identifier 10.1109/JIOT.2025.3589644

2327-4662 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY GUWAHATI. Downloaded on January 27,2026 at 18:16:58 UTC from IEEE Xplore. Restrictions apply.

when it comes to navigating the path planning challenges for UAVs in 3-D spaces riddled with numerous obstacles, heuristic algorithms continue to grapple with issues, such as elevated collision probabilities, sluggish convergence rates, and inadequate precision in convergence [12].

Therefore, this article improves the original PSO and proposes a multistrategy improved PSO algorithm, the MSIPSO algorithm. The main contributions of this article are summarized as follows.

1) To address the challenge of finding feasible solutions in UAV path planning due to dense low-altitude obstacles, a local deadlock escape strategy is proposed. This involves redesigning the velocity update equation of the particle swarm algorithm to enhance randomness, reduce directionality, lift velocity constraints, and enable rapid searching of feasible paths.
2) To overcome the issue of weak algorithm adaptability in 3-D UAV path planning, which results from the use of fixed inertia coefficients and asynchronous learning factors, an adaptive parameter adjustment method is presented. By introducing adaptive nonlinear inertia coefficients and asynchronous learning factors, the stability and convergence performance of the algorithm are enhanced.
3) In response to the inefficiency of local optimal search caused by synchronized particle updates in UAV path planning, multiple population differentiation and evolution strategies are proposed. Different position update equations are designed for particles of varying qualities, thereby improving the search efficiency and overall performance of the algorithm.

The structure of this article is delineated as follows. Section II provides a summary of the existing literature. Section III delves into the complexities of UAV path planning. Section IV introduces the proposed algorithm. Section V details the simulation results and the associated discussions. Finally, Section VI offers a comprehensive summary of this article.

II. Related Works

After decades of development, UAV path planning algorithms have become very rich [13]. According to different principles, it can be summed up into the following categories: 1) graph search based algorithms; 2) sampling based algorithms; and 3) heuristic algorithms. The algorithm based on graph search transforms the constructed model into a topological space [14]. These algorithms mainly include Dijkstra's algorithm, A* algorithm, and D* algorithm. In article [15], a matrix alignment dijkstra method was therefore proposed to find the optimal path through the historical information of the navigator matrix and matrix alignment operations. Zhang et al. [16] introduced a threat function and a variable step size search strategy in the A* algorithm, and improved the search method of the A* algorithm to a sector area search strategy. However, the algorithm based on graph search has the disadvantages of low path smoothness and considerable computational complexity when applied to UAV path planning in 3-D environments [17].

Sampling-based algorithms mainly include fast extended random tree methods (RRT), artificial potential field (APF) methods, and dynamic window approach (DWA) [18]. Usually, such algorithms require map preprocessing and then use a large amount of sampling to search the entire space, in order to find an optimal path. To address the issues of low space utilization and poor smoothness of the path in narrow environments, Chang et al. [19] integrated skeleton extraction and greedy algorithms into the RRT algorithm, enhancing the efficiency and precision of the pathfinding process. However, when applied to outdoor spaces, the advantages of this algorithm are no longer apparent. Long Chen et al. [20] used a dual tree structure to separate the extension and optimization processes of RRT*, improving the convergence speed of the algorithm. However, how to handle multiconstraint problems remains a major challenge for RRT algorithms [21]. Yang et al. [22] proposed an adaptive step size adjustment method based on the predicted distance and the distance and angle relationship between obstacles, which improved the smoothness of APF planning paths and reduced computational complexity. But in dense and chaotic environments, oscillations may occur around obstacles, and even fall into local optima, making APF unsuitable for environments with complex obstacles [23]. The DWA has the problem of insufficient foresight, as it only simulates and evaluates the optimal trajectory within the next window, rather than the global optimal path.

With the increasing complexity of tasks, heuristic algorithms are increasingly being applied to UAV path planning [24]. The idea of heuristic path planning algorithm originates from the behavior or evolutionary laws of natural organisms, and then models and matches the path planning problem with the corresponding natural biological behavior [25]. During the iteration process, it continuously seeks optimization and elimination based on the fitness function, in order to obtain one or a set of high-quality feasible path solutions. WOA is a swarm intelligence optimization algorithm inspired by the hunting behavior of humpback whales, which simulates their social behavior, especially their bubble net strategy during hunting. Shao et al. [26] used quantum behavioral mechanisms to coordinate the evolution of variable stage whale populations, improve population quality, and balance the exploration and development capabilities of WOA. However, WOA has strong dependence on initial solutions and high sensitivity to parameters, especially in complex high-dimensional spaces where it performs poorly [27]. The GWO algorithm simulates the social hierarchy and hunting mechanism of gray wolves, dividing them into four types representing different roles in the wolf pack, with strong convergence performance, few parameters, and easy implementation. Yu et al. [28] proposed a hybrid gray wolf differential evolution algorithm that extends the position update equation and introduces a rank-based mutation method, which enhances the algorithm's ability to jump out of local optima while maintaining fast convergence. However, due to the elite strategy adopted by GWO for population updating, it is more prone to getting stuck in local optima compared to

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY GUWAHATI. Downloaded on January 27,2026 at 18:16:58 UTC from IEEE Xplore. Restrictions apply.

other algorithms when faced with poor initial solutions and complex solution spaces [29].

The PSO algorithm simulates the foraging behavior of bird flocks, treating each solution as a particle in the search space and calculating the evaluation value of each particle through a fitness function [30]. In addition, each particle also has a velocity that determines its direction and distance of motion per unit time. During the iteration process, particles will continuously approach the particle with the best fitness and ultimately obtain the solution result [31]. Ma and Chen [32] proposed an adaptive PSO (APP-PSO) method, which improved the initialization strategy and adopted adaptive dimension and update strategies to achieve adaptive path planning for UAVs in complex environments. In order to solve the problem of premature convergence of PSO algorithm in the face of complex multimodal problems, Zhang [33] proposed an elite contour driven PSO (EAPSO) method, which improves the global search ability of PSO algorithm by establishing three elite contours and calling them to update particle positions. Xu et al. [34] proposed an improved PSO algorithm to solve the path planning problem, which further improves the performance of both dynamic multi group PSO (DMSPSO) and synthetic learning PSO (CLPSO) algorithms by utilizing their ideas. Van Nguyen et al. [35] proposed a Fermat Weber position PSO (FWL-PSO) algorithm. By utilizing the Fermat Weber optimality, a set of elite particles is established, and the Fermat Weber positions of these particles are utilized to replace the traditional global optimum, thereby enhancing the learning strategy of the standard particle swarm algorithm.

To adapt to the 3-D dense obstacle space in low altitude urban environments and quickly plan the shortest path of UAVs, this study models the 3-D path planning problem in low altitude urban environments and designs the MSIPSO algorithm based on the characteristics of the problem. Compared with existing algorithms, MSIPSO has higher stability, faster convergence speed, and better optimization ability.

III. UAV Path Planning

A. Environment Modeling

The rapid development of the IoT has provided a technological foundation for digital city modeling. Through the extensive deployment of sensor networks in urban environments, IoT can collect multidimensional data in real time, including terrain, weather conditions, traffic flow, and environmental quality. This data provides high-precision input information for digital city modeling, enabling urban models to more realistically reflect the real environment. In addition, IoT technology also supports low-latency communication between drones and ground control centers, ensuring that path planning algorithms can make decisions quickly based on real-time data.

However, precise data and real-time communication technologies alone are insufficient. As depicted in Fig. 1, in the low-altitude urban environment, the dense building clusters increase the difficulty for UAVs to find the optimal path. To find a feasible path, path planning algorithms need to maintain a broad search capability while also exploring the entire space more meticulously. To address this challenge, it is crucial to establish appropriate environmental and problem models before planning the path.

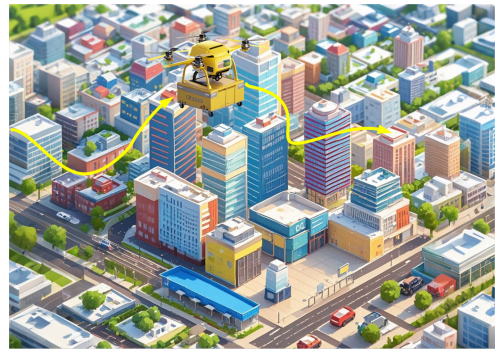

Fig. 1. Schematic of UAV logistics distribution.

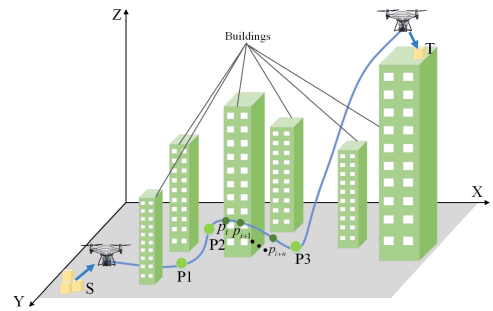

Fig. 2. Schematic of UAV path planning.

Therefore, as shown in Fig. 2, this work does not employ the conventional and more conservative cylindrical modeling, but instead adopts cubic modeling to represent obstacles. Compared to cylindrical modeling, cubic modeling better matches the morphological characteristics of urban buildings and has higher spatial utilization. From the perspective of the solution space, this approach offers UAVs more feasible paths. UAV path planning aims to find a collision free, shortest, and smooth path between the starting point and the target point. This is an optimization problem. Therefore, this work sets control points P1, P2, and P3 as the carriers of the optimization objectives, which can be regarded as particles in MSIPSO. According to B-splines, a smooth curve with coordinates can be generated through these three control points, as well as the starting point S and the target point T. By optimizing the positions of P1, P2, and P3 through MSIPSO, the optimal path can be found.

B. Constraint Condition and Cost Function

UAVs need to fully consider constraints, such as path length, path safety, and flight altitude during flight, and achieve optimal path planning for UAV by designing a total cost function. When executing logistics tasks in low altitude urban environments, the main considerations for constructing a total cost function are path length cost, path safety cost, flight height cost, and angle change cost, as follows.

1) Path Length Cost: The path length cost f_L is composed of the distance between two adjacent nodes, and its calculation

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY GUWAHATI. Downloaded on January 27,2026 at 18:16:58 UTC from IEEE Xplore. Restrictions apply.

formula is shown in

$$f_L = \sum_{i=1}^{n-1} \sqrt{(x_i - x_{i+1})^2 + (y_i - y_{i+1})^2 + (z_i - z_{i+1})^2} \quad (1)$$

where n is the total number of discrete points generated by these control points.

2) Path Safety Cost: By using the penalty coefficient method, a safety cost function f_S is constructed. With an increase in the frequency of collisions between the path and obstacles, the associated security costs rise proportionally, emphasizing the importance of choosing a path that avoids obstacles during the path planning process. Its calculation formula is shown in

$$f_S = c \times K_{\max} \times f_L. \quad (2)$$

Among them, c is the number of times the path crosses obstacles, $K_{\max}$ is the penalty coefficient, and f_L is the path length.

3) Flight Height Cost: During UAVs flight, the flight height is typically limited between two given extreme values. And UAVs are expected to fly at a stable height. The height cost calculation formula is shown in

$$H_i = \begin{cases} K_{\max} & \text{if } z_i < h_{\min} \text{ or } z_i \geq h_{\max} \\ \frac{|h_{\max} - h_{\min}|}{4z_i} & \text{if } h_{\min} \leq z_i < \frac{h_{\max}+h_{\min}}{2} \\ \frac{|z_i - h_{\max} + h_{\min}|}{4} & \text{if } \frac{h_{\max}+h_{\min}}{2} \leq z_i < h_{\max}. \end{cases} \quad (3)$$

Among them, Z_i represents the true height of UAV. In this work, the direction of algorithm optimization is toward the minimization of the cost function, hence particles tend to avoid points with higher cost values. Given this premise, when the height of a path point exceeds the maximum allowable height or falls below the minimum allowable height, H_i is assigned a very large value, $K_{\max}$. This results in a significantly high cost for that point, causing particles to quickly move away from it.

When the height of a path point lies between the minimum and intermediate heights, a decreasing function is designed to calculate the cost. As the flight altitude Z_i decreases (i.e., the UAV flies closer to the ground), the cost increases due to the decreasing function. Conversely, as Z_i increases (i.e., the UAV approaches the intermediate layer), the cost decreases, prompting particles to move toward the intermediate layer.

When the height of a path point is between the maximum and intermediate heights, an increasing function is designed for the cost. Similar to particles below the intermediate height, this function ensures that as optimization progresses, particles are guided toward the intermediate layer.

By designing the flight altitude penalty factor in this way, UAVs are discouraged from flying in the low-altitude areas with many unknown dangers and the high-altitude areas with unstable air currents. Instead, they are encouraged to fly within the medium-altitude airspace, where the air currents are more stable and the flight is safer. The height cost f_H is obtained by adding H_i of all path nodes, and its calculation formula is shown in

$$f_H = \sum_{i=1}^{n} H_i. \quad (4)$$

4) Angle Change Cost: Excessive pitch and yaw angles can affect the stability of UAV flight and increase the risk of cargo dropping. Therefore, the pitch and yaw angles of the UAV should be limited to a reasonable range and kept as small as possible. The cost function f_A is as follows:

$$f_A = f_{\text{pit}} + f_{\text{yaw}} \quad (5)$$

$$f_{\text{pit}} = \begin{cases} K_{\max}\alpha_i > \alpha_{\max} \\ \sum_{i=1}^{n-1} \alpha_i \alpha_i \leq \alpha_{\max} \end{cases} \quad (6)$$

$$f_{\text{yaw}} = \begin{cases} K_{\max}\beta_i > \beta_{\max} \\ \sum_{i=1}^{n-1} \beta_i \beta_i \leq \beta_{\max} \end{cases} \quad (7)$$

$$\alpha_i = \left| \arctan\left(\frac{z_{i+1} - z_i}{x_{i+1} - x_i} \right) \right|, \quad i = 1, 2, 3, \ldots, n-1 \quad (8)$$

$$\beta_i = \left| \arctan\left(\frac{y_{i+1} - y_i}{x_{i+1} - x_i} \right) \right|, \quad i = 1, 2, 3, \ldots, n-1. \quad (9)$$

Among them, α_i and β_i are the horizontal and vertical angles of the *ith* discrete point, $\alpha_{\max}$ and $\beta_{\max}$ are the maximum horizontal and vertical angles, and are the horizontal and vertical angle costs, and is the total number of discrete points on the path.

Finally, this study comprehensively integrates the aforementioned UAV path evaluation metrics to construct a total fitness function. The mathematical formulation of the fitness function is given by

$$F = \lambda f_L + \mu f_S + \phi f_H + \gamma f_A \quad (10)$$

where f_L, f_S, f_H, and f_A represent the cost functions for path length, path safety, flight height and angle change cost, respectively, as previously defined. λ, μ, ϕ, and γ denote the weighting coefficients for each cost component. By adjusting these weighting coefficients, the path planning algorithm can prioritize different operational requirements. The proposed algorithm optimizes the solution by minimizing the fitness function F, thereby generating an optimal path that satisfies multiple performance criteria.

IV. Proposed MSIPSO Algorithm

A. PSO Algorithm

PSO algorithm is a type of swarm intelligence algorithm with wide applications in optimization problems. The mathematical expression for updating particle position and velocity is as follows:

$$p_i^{t+1} = p_i^t + v_i^{t+1} \quad (11)$$

$$v_i^{t+1} = w v_i^t + c_1 r_1 \left(s_i^t - p_i^t\right) + c_2 r_2 \left(g^t - p_i^t\right) \quad (12)$$

where w is an inertia weight parameter; The inertia weight governs the extent to which a particle retains its historical velocity during the search process, thereby influencing the global and local search capabilities of the PSO algorithm. A larger inertia weight w facilitates the preservation of particle motion momentum, enabling particles to explore with higher velocities. This manifests as pronounced oscillations that enhance global search capability by escaping local optima. Conversely, a smaller w accelerates algorithm convergence and strengthens local search performance through intensified

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY GUWAHATI. Downloaded on January 27,2026 at 18:16:58 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 PSO Algorithm

Initialization: Particle position p_i^t and velocity v_i^t, individual optimal position s_i, historical optimal position g^t.
while $t < M$ **do**: $t = t + 1$
  for $i = 1{:}N$ **do**:
    Update the velocity v_i^t of the particle using Eq. (12)
    Update the position p_i^t of the particle using Eq. (11)
    Calculate the fitness value F_i^t using Eq. (10)
    Update the individual historical optimal position s_i^t
  end for
  Update the global historical optimal position g^t
end while
return the historical optimal position g^t as the optimal solution

particle aggregation. However, excessive w values may prevent convergence to precise solutions, while insufficient w values risk premature convergence to local optima. Therefore, the inertia weight should be appropriately adjusted according to the specific characteristics of the task at hand to balance exploration-exploitation tradeoffs.

c_1 and c_2 are learning factors that control the learning speed of particles for individual optimality and collective optimality, respectively; These parameters govern the particle's trust level in individual versus collective experience. The cognitive coefficient c_1 represents the particle's self-awareness, while the social coefficient c_2 embodies its collective consciousness. During iteration, higher relative c_1 values strengthen particle individualism, manifesting as dispersed search patterns with slower convergence rates and potential failure to converge. Conversely, elevated c_2 values accelerate swarm consensus formation at the risk of premature convergence, particularly problematic when combined with small w and c_1 values - under such conditions, particles rapidly converge toward the initial global best position without adequate exploration. r_1 and r_2 are two random numbers in [0, 1]; s_i^t and g^t are historical optimal position of individual and population, respectively.

Fig. 3 shows the update process of particles. Particles update based on their current position, individual optimal position, and global optimal position.

B. Multi Strategy Improved Particle Swarm Optimization Algorithm

The PSO algorithm has strong optimization capabilities, but when applied to UAV path planning in low altitude environments with dense obstacles, it is prone to local deadlocks, difficult parameter adjustments, and low search efficiency. In response to the above shortcomings, the MSIPSO algorithm has been proposed. The original PSO algorithm is improved through the following three strategies, which will enhance the comprehensive performance of the algorithm in solving UAV path planning problems in low altitude urban environments.

1) Local Deadlock Escape Strategy: As discussed earlier, in high-dimensional dense obstacle environments, using the original PSO to solve path planning can easily lead to long-term unsolvable states, namely, local deadlocks. This is caused by the poor initial position of the particle swarm. In this state,

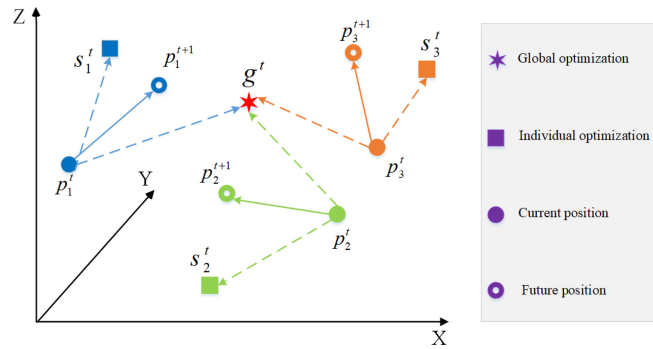

Fig. 3. Update process of particles in PSO.

Algorithm 2 Local Deadlock Jump Strategy

Initialization: Fitness threshold F_{max};
if $F(g^t) > F_{max}$ **then**
  for $i = 1{:}N$ **do**
    Release the velocity boundary constraint;
    Update the velocity v_i^t of the particle using Eq. (14);
    Update the position p_i^t of the particle using Eq. (11);
    Calculate the fitness value F_i^t using Eq. (10);
    Update the individual historical optimal position s_i^t;
  end for
  Update the global historical optimal position g^t;
end if
return the position p_i^{t+1} of the particle

each generation's optimal path has some segments that overlap with obstacles. On the iteration curve, this manifests as the optimal fitness of each generation remaining at a very high value as the iterations progress. This occurs because the path safety-related cost function f_S in the objective function design incorporates an extremely large penalty value $K_{\max}$. Therefore, this article introduces a fitness threshold $F_{\max}$ to identify such a state. $F_{\max}$ is calculated as follows:

$$F_{\max} = K_{\max} \cdot \|\mathbf{p}_{\text{end}} - \mathbf{p}_{\text{start}}\| \tag{13}$$

where $\mathbf{p}_{\text{start}}$ and $\mathbf{p}_{\text{end}}$ are the starting vector and ending vector, respectively. When the tth generation global best fitness $F(g^t) > F_{\max}$, the algorithm is considered trapped in a local deadlock state, triggering the local deadlock escape strategy. Its specific steps are as Algorithm 2.

First, to prevent optimization capability loss due to overly large particle velocities, the original PSO conventionally enforces velocity boundary constraints. However, in the local deadlock state, the main task is to quickly find feasible solutions. Therefore, in the local deadlock state, MSIPSO releases the velocity boundary constraint, allowing particles to quickly search the entire solution space at higher speeds.

Second, from (12) and Fig. 3, it can be seen that the update of the particle is directional, determined by the current particle position, individual historical best particle and global historical best particle. But in a local deadlock state, the search for particles is expected to be highly random rather than strongly

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY GUWAHATI. Downloaded on January 27,2026 at 18:16:58 UTC from IEEE Xplore. Restrictions apply.

directional. Therefore, the MSIPSO algorithm has redesigned the velocity update formula for local deadlock states, as shown

$$v_i^{t+1} = A - 2A \cdot r_3 \tag{14}$$

$$A = \sqrt{\frac{L}{N}} \tag{15}$$

where A is a constant determined by the size of the flight space and number of particles. L is the length of a certain dimension in the flight space, and N is the number of particles. r_3 is random number in [0, 1]. Unlike the original directional position updates, the introduction of high randomness empowers each particle to freely explore the entire search space. Particles no longer update solely near the current optimal solution but instead randomly probe other regions. This enables the swarm to quickly escape local deadlock states caused by poor initial solutions and transition into the optimization phase upon discovering feasible solutions.

Overall, the measures of eliminating the speed boundary restriction and rewriting the speed update equation save iteration resources for the entire search process and reserved sufficient iteration rounds for subsequent optimization stages. And it enables the algorithm to timely escape from local deadlock states, improving the stability of the algorithm when facing dense and complex environments.

2) Adaptive Inertia Coefficient and Learning Factors: As described in Section IV-A, in PSO, inertia weight, individual learning factor, and social learning factor are three key parameters that have a significant impact on the algorithm's search ability and convergence performance.

For UAV path planning missions in dense urban environments, the particle swarm is required to prioritize extensive spatial exploration during the initial phase to rapidly identify feasible solutions, emphasizing oscillatory behavior, while demonstrating convergence characteristics toward theoretically optimal positions during later stages with intensified aggregation. This process should ideally incorporate nonlinear transition properties. Conventional fixed parameter configurations prove inadequate to satisfy these dynamic requirements, as identical coefficient settings may exhibit substantially different efficacy across varied environments. Furthermore, as illustrated in Section IV-A, originally appropriate parameters may become suboptimal or even counterproductive as mission priorities evolve through iterative progression, particularly when addressing the shifting balance between exploration and exploitation throughout optimization cycles. Therefore, MSIPSO introduces an exponential nonlinear decreasing inertia coefficient and a logarithmic nonlinear asynchronous learning factor. The specific mathematical expression is as follows:

$$w(t) = w_{\min} + (w_{\max} - w_{\min}) \cdot \frac{e^{-\frac{t}{T}} - e^{-1}}{e^{-\frac{1}{T}} - e^{-1}} \tag{16}$$

$$c_1(t) = c_{1_{\max}} - \left(c_{1_{\max}} - c_{1_{\min}}\right) \cdot \ln\left(1 + (e-1) \cdot \frac{t}{T}\right) \tag{17}$$

$$c_2(t) = c_{2_{\min}} + \left(c_{2_{\max}} - c_{2_{\min}}\right) \cdot \ln\left(1 + (e-1) \cdot \frac{t}{T}\right) \tag{18}$$

where, t denotes the current iteration count, and T represents the maximum iteration count. w is the inertia coefficient, with $w_{\min}$ and $w_{\max}$ being the lower and upper bounds of the inertia weight, respectively. The nonlinear factor $e^{-(t/T)}$, derived from the properties of the exponential function, is a monotonically decreasing function whose value gradually diminishes as the independent variable t (i.e., the iteration count) increases. $([e^{-t/T} - e^{-1}]/[e^{-1/T} - e^{-1}])$ linearly maps the value of $e^{-(t/T)}$ within the range [0, 1] while preserving its nonlinear characteristics. When $t = 1$,

$$w(1) = w_{\min} + (w_{\max} - w_{\min}) \cdot \frac{e^{-1/T} - e^{-1}}{e^{-1/T} - e^{-1}} = w_{\max}$$

when $t = T$,

$$w(T) = w_{\min} + (w_{\max} - w_{\min}) \cdot \frac{e^{-1} - e^{-1}}{e^{-1/T} - e^{-1}} = w_{\min}.$$

As t increases from 1 to T, w nonlinearly decreases from $w_{\max}$ to $w_{\min}$. This design aims to maintain a dynamic balance between the exploration and optimization at different stages. The inertia weight w reflects the particle's ability to retain the inertia of its historical velocity.

In the early stage of iteration, w is relatively large. Combining (12), it can be concluded that the particle update speed is high. Based on (11), it can be seen that the particle position is updated with a large step size in the search space, and the particle swarm can quickly search the entire space. On the iteration curve, this is manifested as the rapid convergence of the algorithm in the early stage. As the iteration progresses, w gradually decreases, the particle update speed decreases, and the algorithm transitions from the exploration stage to the optimization stage, improving the convergence accuracy of the algorithm.

In addition, the exponential formula of (16) forms a concave function in geometric mathematics, which ensures that the value of w accelerates its decline in the early stages of iteration and then stabilizes in the later stages. This nonlinear attenuation characteristic effectively prevents excessive accumulation of historical velocity in the early exploration stage, while reserving sufficient time resources for accuracy optimization, further ensuring the convergence accuracy of the algorithm.

c_1 and c_2 are individual learning factors and social learning factors, respectively. They control the step size at which particles move toward their individual historical best position and global best position. $c_{1\min}$ and $c_{2\min}$ are the lower bounds of individual learning factors and social learning factors, respectively. $c_{1\max}$ and $c_{2\max}$ are the upper bounds of individual learning factors and social learning factors, respectively. Equations (17) and (18) formulate an asynchronous update strategy for learning factors based on task-specific characteristics of path planning. Specifically, during initial search phases, the algorithm employs elevated c_1 values coupled with reduced c_2 magnitudes to emphasize particle autonomy for enhanced global exploration. As iterations progress, c_1 undergoes nonlinear attenuation while c_2 demonstrates complementary nonlinear amplification, dynamically shifting optimization emphasis toward swarm consensus to strengthen convergence performance toward the global optimum.

3) Diversified Evolutionary Strategies: In general, the detection and development capabilities of particles are contradictory. That is, a particle can only exist in one state in an

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY GUWAHATI. Downloaded on January 27,2026 at 18:16:58 UTC from IEEE Xplore. Restrictions apply.

Algorithm 3 Diversified Evolutionary Strategy

Initialization: Fitness threshold F_{max}, population classification constant E, a and b;
if $F(g^t) < F_{max}$ **then**
  Sort particles by fitness value;
  for $i = 1{:}\frac{aM}{E}$ **do**
    Update the velocity v_i^t of the particle using Eq. (12);
    Update the position p_i^t of the particle using Eq. (18);
  end for
  for $i = \frac{aM}{E}{:}\frac{(E-a-b)}{E}$ **do**
    Update the velocity v_i^t of the particle using Eq. (12);
    Update the position p_i^t of the particle using Eq. (19);
  end for
  for $i = \frac{(E-a-b)}{E}{:}N$ **do**
    Update the velocity v_i^t of the particle using Eq. (12);
    Update the position p_i^t of the particle using Eq. (20);
  end for
  Calculate the fitness value F_i^t using Eq. (10);
  Update the individual historical optimal position s_i^t;
  Update the global historical optimal position g^t;
end if
return the position p_i^{t+1} of the particle

iteration period, either with strong divergence or strong convergence. Therefore, in order to coordinate the detection and development capabilities of particles in the PSO optimization process, a multi swarm approach is proposed for differentiated search. The basic idea is to divide the entire population into multiple subgroups based on their fitness level, and allocate different particle update strategies according to the quality of the population.

This article divides the entire population into three subgroups, represented by D_i. Subgroup D_1 is the advanced group closest to the current optimal solution. Therefore, a refined search strategy was adopted to find the optimal solution with higher accuracy. Subgroup D_2 has a large number and adopts a large-scale search strategy to quickly search the entire solution space. Subgroup D_3 is far away from the current optimal solution and adopts a random update strategy to increase population diversity and avoid the system falling into local optima. Algorithm 3 displays the classification criteria and corresponding update strategies for three types of populations.

The mathematical description of the three types of population update strategies is as follows.

For particles in the higher population D_1

$$p_i^{D_1}(t+1) = p_i^{D_1}(t) + \left(a_1 + (a_2 - a_1) \cdot \frac{t}{M}\right) \cdot v_i(t+1). \quad (19)$$

For particles in the medium population D_2

$$p_i^{D_2}(t+1) = p_i^{D_2}(t) + \left(b_1 + (b_2 - b_1) \cdot \frac{t}{M}\right) \cdot v_i(t+1). \quad (20)$$

For particles in the low level population D_3

$$p_i^{D_3}(t+1) = r4 \cdot (p_{\max} - p_{\min}) \quad (21)$$

where a_1 and a_2 are the upper and lower bounds of the velocity weight coefficients for D_1 populations, respectively. b_1 and b_2 are the upper and lower bounds of the velocity weight coefficients for D_2 populations, respectively. And they satisfy $a_1 < b_1$, $a_2 < b_2$ and $(a_2 - a_1) < (b_2 - b_1)$. Through such mathematical constraints, the elite population D_1 can be guaranteed to perform local refinement optimization around the current optimal solution at a low rate to find a better solution. The medium population D_2 can search the whole space at relatively large speed to find the possibility beyond the current optimal for the population. $p_{\max}$ and $p_{\min}$ are the upper and lower bounds of particle position, containing three dimensions: x, y, and z. r_3 is random number in [0, 1]. Equation (20) introduces randomization for D_3 population. This not only removes a few bad particles for the entire population, but also increases the diversity of the population.

C. Complexity Analysis

1) Time Complexity Analysis: Based on pseudo-code and algorithm execution process, the time complexity of MSIPSO is analyzed as follows:

$$\begin{aligned} O(T) &= O(\text{Initialization}) + O(\text{sort}) + O(\text{Fitness calculation}) \\ &\quad + O(\text{Update velocity}) + O(\text{Update location}) \\ &\quad + O(\text{Update global optimum}) \\ &= O(2 \times N \times D) + O(T \times N \times \log N) \\ &\quad + O(T \times N \times F) + O(T \times N \times D \times f_v) \\ &\quad + O\big(T \times N \times D \times f_p\big) + O(T \times D) \\ &= O(T \times N \times D) \end{aligned} \quad (22)$$

where N denote the number of particles in the swarm, D represent the dimensionality of the search space, T indicate the maximum number of iterations, F correspond to the time required for a single particle to compute the fitness function, f_v signify the time needed to update a particle's velocity, and f_p denote the time consumed in updating a particle's position.

The local deadlock escape strategy introduces a mutex operation into the original algorithm without affecting the asymptotic time complexity. The adaptive parameter strategy only alters the composition of constant factors, leaving the asymptotic complexity order unchanged. For the cooperative evolution strategy, although sorting operations are introduced, the improved convergence rate compensates for this addition. Moreover, compared with the computationally intensive particle update loop, the associated time complexity is relatively small and can be considered negligible. As documented in [36], The time complexity of PSO GWO and WOA is $O(T \times N \times D)$. By analyzing the structure of elite archive-driven PSO (EAPSO) and hybrid GWO and differential evolution (HGWODE), we can see that their time complexity is also $O(T \times N \times D)$ [28], [33]. Therefore, the proposed MSIPSO has not introduced significant computational overhead in time complexity.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY GUWAHATI. Downloaded on January 27,2026 at 18:16:58 UTC from IEEE Xplore. Restrictions apply.

Algorithm 4 MSIPSO

Income: Starting point $S(x_1, y_1, z_1)$, target point $T(x_2, y_2, z_2)$.
Outcome: Discretized path points.
Initialization: Particle position p_i^t and velocity v_i^t, individual optimal position s_i, historical optimal position g^t,fitness threshold F_{max} ,population classification constant E, a and b;
while $t < M$ **do**
  $t = t + 1$
  Calculate the inertia coefficient w using Eq. (15);
  Calculate the individual learning factor c_1 using Eq. (16);
  Calculate the social learning factor c_2 using Eq. (17);
  for $i = 1$: N **do**
    if $F(g^t) \geq F_{max}$ **then**
      /* Local deadlock jump strategy */
    end if
    if $F(g^t) < F_{max}$ **then**
      /* Diversified evolutionary strategies for multiple populations */
    end if
    Calculate the fitness value F_i^t using Eq. (10);
    if $F_i^t > F_i^{t-1}$ **then**
      Update the individual historical optimal position s_i^t;
    end if
  end for
  if new global best is found **then**
    Update the global historical optimal position g^t;
  end if
end while
return the best path.

2) Space Complexity Analysis: The proposed MSIPSO necessitates storage allocations for particle positions, velocities, sequencing parameters, personal best solutions, and the global best solution. The corresponding space complexity can be formally derived as follows:

$$\begin{aligned} O(S) &= O(\text{velocity}) + O(\text{location}) + O(\text{sequence}) \\ &\quad + O(\text{Individual optimal location}) \\ &\quad + O(\text{Global optimal location}) \\ &= O(N \times D) + O(N \times D) + O(N) \\ &\quad + O(D) + O(D) \\ &= O(N \times D). \end{aligned} \tag{23}$$

According to article [36] and the structure of the original PSO algorithm, the spatial complexity of PSO is as follows:

$$\begin{aligned} O_S(PSO) &= O(\text{velocity}) + O(\text{location}) \\ &\quad + O(\text{Individual optimal location}) \\ &\quad + O(\text{Global optimal location}) \\ &= O(N \times D) + O(N \times D) \\ &\quad + O(D) + O(D) \\ &= O(N \times D). \end{aligned} \tag{24}$$

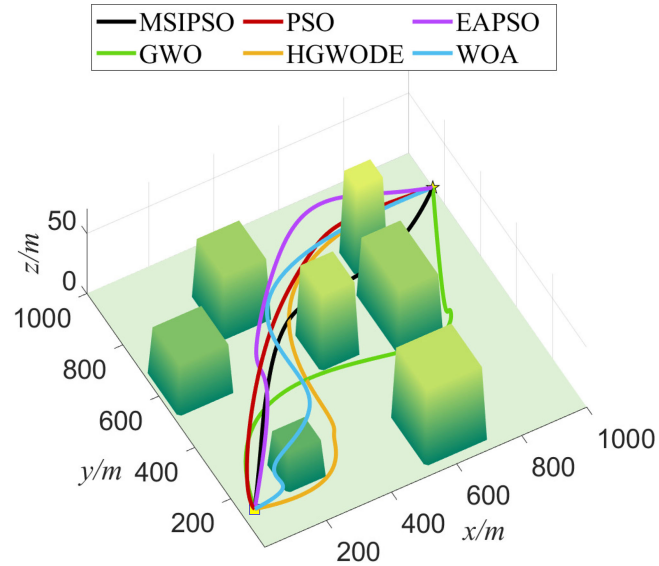

Fig. 4. Path planning for UAV using various algorithms in a simple environment (side view).

To sum up, MSIPSO and PSO have the same complexity. Similarly, by analyzing the algorithm structure of GWO WOA EAPSO and HGWODE, some mathematically negligible items have little impact on the actual space occupation [28], [33], [36]. They have not significantly increased the storage occupation, and their space complexity is also $O(N \times D)$. In the next section, this article will verify the performance of MSIPSO based on three different density simulation environments.

V. Simulation and Comparative Analysis

In order to verify the superiority of MSIPSO in UAV path planning tasks in low altitude urban scenarios, the proposed algorithm was compared and analyzed with other algorithms in three simulation environments. The flight space size of the three 3-D scenes is the same, but the difference lies in the size and distribution of obstacles. The size of the flight space is set to [1000m × 1000m × 120m], with starting and ending points of [40, 120, 5] and [980, 800, 10], respectively. In this study, the decision variables were three control points. The B-spline curve is generated through three control points, as well as a starting point and a target point. The maximum number of iterations is set to 100. The hardware used for implementation is 32 GB of memory on 12th Gen Intel Core i7-12700 and MATLAB 2024a. All six algorithms are run under the same hardware conditions. The simulation in all environments was repeated 100 times, and the optimal fitness value, convergence time, and other experimental results were recorded during the experiment.

A. Simple Environment

In order to simulate a sparse obstacle environment, seven obstacles of different sizes and positions were set in the search space. In a simple environment, MSIPSO and five other algorithms are applied to UAV path planning, as shown in Figs. 4 and 5. It can be seen that all six algorithms have

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY GUWAHATI. Downloaded on January 27,2026 at 18:16:58 UTC from IEEE Xplore. Restrictions apply.

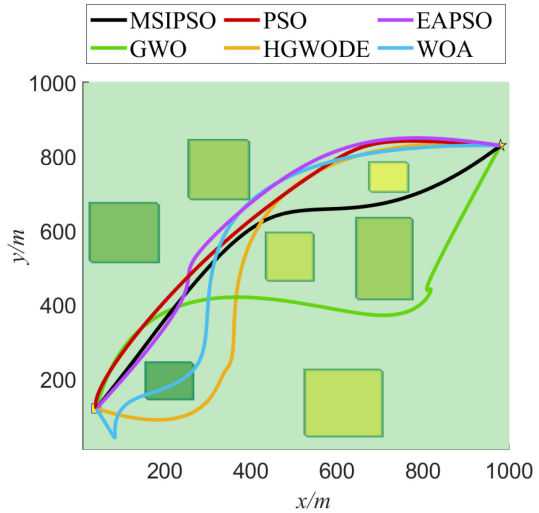

Fig. 5. Path planning for UAV using various algorithms in a simple environment (top view).

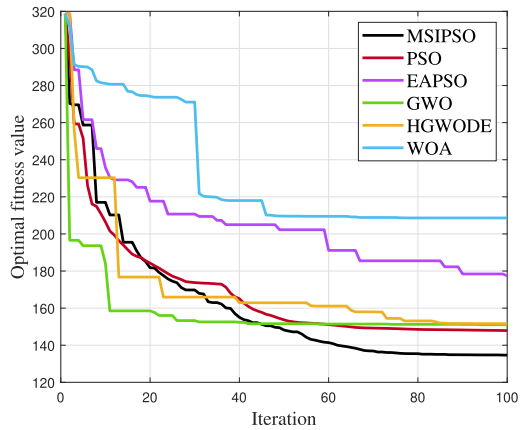

Fig. 6. Optimal fitness values evolve throughout the iterations within a simple environment.

successfully planned feasible paths. However, as shown in Fig. 6, MSIPSO achieved the lowest fitness value in terms of convergence accuracy. This means that MSIPSO has found a better path. From the perspective of convergence speed, in the early stages of the algorithm, GWO algorithm and its improved algorithms have the fastest convergence speed, which is determined by the elite leadership method used by GWO algorithm to update the wolf pack position. But from a global perspective, the convergence achieved by MSIPSO is better. Even though some algorithms found the so-called optimal solution as early as the 25th generation, but MSIPSO found a better solution than other algorithms in the 45th generation, and maintained this advantage in the next iteration. Therefore, overall, compared to other algorithms, the MSIPSO algorithm performs the best in scenario 1.

Afterwards, in order to eliminate accidental errors, the experiment was repeated 100 times under the same simulation conditions. From Table I, it can be seen that the fluctuation range of the optimal fitness value for MSIPSO is the smallest, ranging from 132 to 140, while the variation range of the optimal fitness values for HGWODE, EAPSO, PSO, WOA and GWO are [140,159], [134,149], [150,169], [135,164], and [145,159], respectively, indicating that MSIPSO has stronger stability in simple environments. Furthermore, the oscillations of MSIPSO within a narrower numerical domain also substantiate its superiority in convergence precision. The comparison between the mean and median further confirms the above conclusion.

TABLE I
COMPARISON OF STATISTICAL OPTIMAL FITNESS VALUE IN THE SIMPLE ENVIRONMENT

Algorithm	Maximum	Minimum	Average	Median
MSIPSO	**140.7178**	**132.0104**	**136.4359**	**136.4619**
PSO	159.9898	140.2067	149.0976	149.0208
EAPSO	149.8392	134.0084	142.3670	143.0043
GWO	169.7587	150.4107	160.2419	160.2219
HGWODE	164.8111	135.5873	150.0310	150.1440
WOA	159.5591	145.1173	151.4642	151.0099

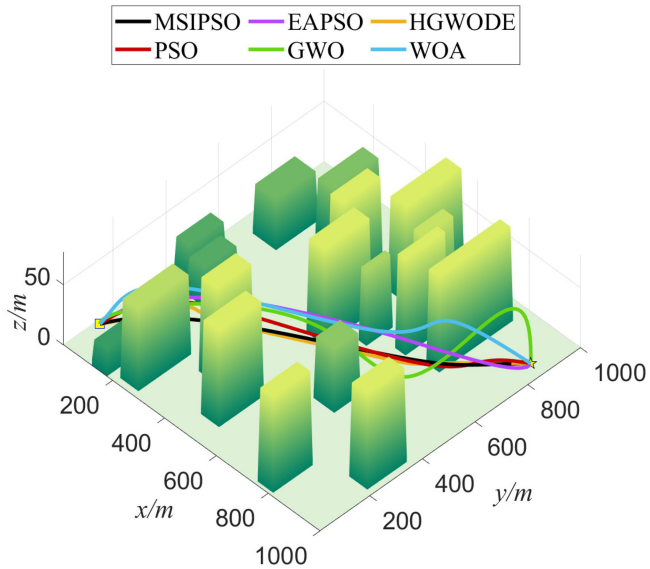

Fig. 7. Path planning for UAV using various algorithms in a normal environment (side view).

B. Normal Environment

In this section, additional obstacles are incorporated into the search space to simulate moderate-density urban environments. These include elongated obstacles and the narrow passages formed between them, which introduce specific challenges for algorithm convergence. Figs. 7 and 8 present comparative path planning results from six algorithms. While all algorithms successfully identified feasible paths, some exhibited increased path lengths and turning angles due to interference from obstacles near the start and target positions. This characteristic is more distinctly revealed through the fitness variation patterns shown in Fig. 9. Notably, the MSIPSO algorithm demonstrates accelerated convergence in complex environments compared to simpler scenarios, achieving stable convergence around the 30th generation-tied for first place with PSO. In terms of convergence precision, MSIPSO maintains its superiority, securing the top position among the six comparative algorithms. Statistical data from 100 repeated experiments

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY GUWAHATI. Downloaded on January 27,2026 at 18:16:58 UTC from IEEE Xplore. Restrictions apply.

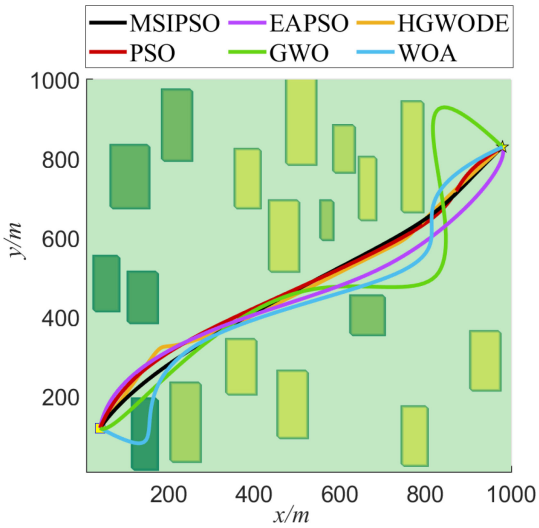

Fig. 8. Path planning for UAV using various algorithms in a normal environment (top view).

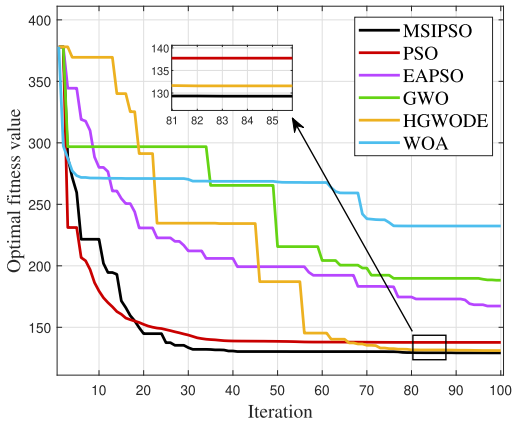

Fig. 9. Optimal fitness values evolve throughout the iterations within a normal environment.

TABLE II
COMPARISON OF STATISTICAL DATAS IN THE NORMAL ENVIRONMENT

Algorithm	Maximum	Minimum	Average	Median
MSIPSO	**134.982**	**129.1091**	**132.0883**	**132.2094**
PSO	164.9213	140.2920	153.5226	153.5807
EAPSO	154.7716	140.2889	148.0027	148.3175
GWO	179.8870	168.1751	155.4169	168.3427
HGWODE	159.9317	132.2745	146.6123	147.4219
WOA	169.9896	150.0284	159.9356	159.2844

(Table II) confirm the algorithm's stability in this experimental configuration. Moreover, the performance gap between the proposed algorithm and other methods in convergence precision shows an expanding trend compared with results obtained in simpler environments.

C. Dense Environment

In order to simulate the low altitude environment of cities with dense obstacles, more obstacles are added to the flight space. Compared to Environment 1 and 2, the flying space has become narrower, and the probability of collision between paths and obstacles has greatly increased. From Fig. 10 (a) and (b), it can be seen that in this environment, some of the algorithms' planned paths collided with obstacles. From the fitness changes in Fig. 10(c), it can be seen that these algorithms have fallen into a local deadlock state. However, due to the design of a local deadlock escape strategy, MSIPSO quickly found the optimal feasible solution and escaped from this state. From the perspective of convergence performance, whether some algorithms fall into local deadlocks or all algorithms are optimized normally as Fig.11, MSIPSO has the best convergence speed and accuracy. This is due to the combined effect of differential evolution strategy and adaptive coefficient.

TABLE III
COMPARISON OF STATISTICAL DATAS IN THE DENSE ENVIRONMENT

Algorithm	Maximum	Minimum	Average	Median	collision
MSIPSO	**144.8756**	**129.0051**	**137.9278**	**138.1727**	**2**
PSO	198.8338	138.0815	170.5604	170.1771	17
EAPSO	199.7229	141.8617	171.0855	172.3685	12
GWO	258.1692	170.5088	214.6596	214.5901	18
HGWODE	249.9329	144.5834	204.4240	207.2605	14
WOA	268.3170	234.5290	201.4433	236.9882	19

Table III records the statistical data of 100 experiments conducted in environments with dense obstacles. For the sake of fairness, the calculation of the maximum, average, and median values excluded the values where the algorithm got stuck in local deadlocks, and recorded the number of collisions between the planned path and obstacles in 100 experiments. From Table III, it can be seen that MSIPSO has the least number of collisions, the smallest fluctuation range and amplitude, which proves the stability of the algorithm. The comparison between the maximum, average, and median values shows the excellent optimization ability of MSIPSO.

D. Parameter Sensitivity Analysis Experiment

Although adaptive parameter strategies can solve some difficulties in parameter selection, boundary parameters, such as $w_{\max}$, $w_{\min}$, $c_{1_{\max}}$, $c_{2_{\max}}$, $c_{1_{\min}}$, and $c_{2_{\min}}$ still need to be manually set. Therefore, the following experiment will conduct parameter sensitivity analysis on MSIPSO based on the single variable method. To ensure algorithm convergence, the following constraints must be satisfied $w < 1$, $c_1 > 0$, $c_2 > 0$, $c_1 + c_2 < 4(w+1)$ [37]. Under these conditions, MSIPSO was tested in dense environments with 50 repeated experiments for each parameter combination. The convergent iteration (the first iteration where fitness deviates from the optimal by less than 5% and optimal fitness values were recorded.

First, four different sets of $w_{\max}$ and $w_{\min}$ were selected to compare the impact of the inertia weight range on algorithm performance. The specific values and performance of each set of parameters are shown in Fig. 12 and Table IV. To ensure a single variable, the relevant parameters of the learning factor are set to fixed values $c_{1_{\min}} = c_{2_{\min}} = 0.5$, $c_{1_{\max}} = c_{2_{\max}} = 2.5$ [38].

Based on the data in Fig. 12 and Table IV, the algorithm performs the best when $w = [0.4, 0.9]$, with the lowest

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY GUWAHATI. Downloaded on January 27,2026 at 18:16:58 UTC from IEEE Xplore. Restrictions apply.

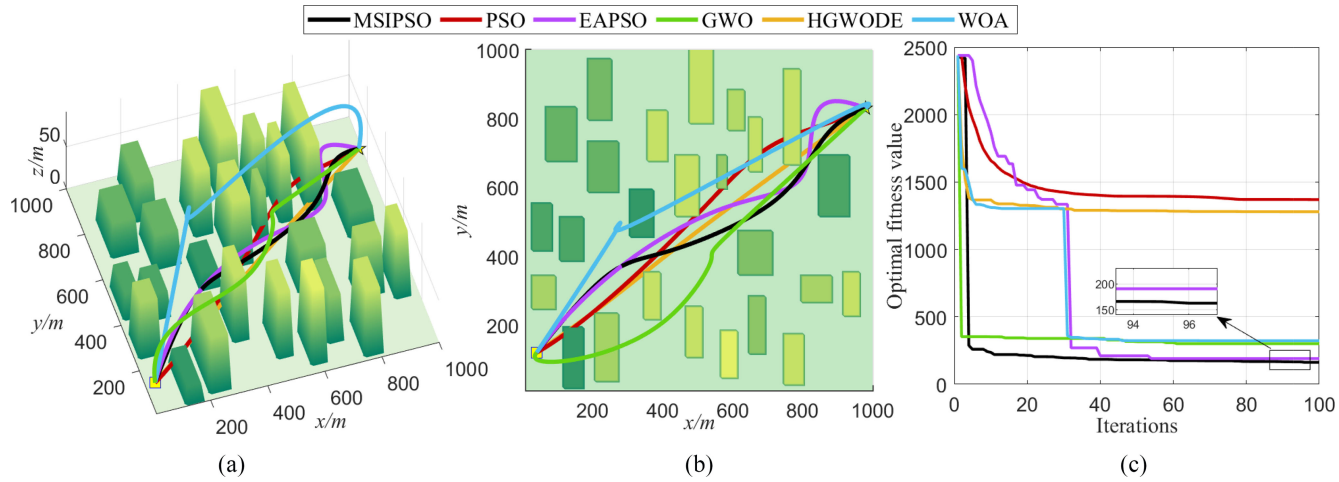

Fig. 10. Comparison of six algorithms in dense environments (scenario 1). (a) 3-D path. (b) Top view. (c) Optimal fitness with iteration.

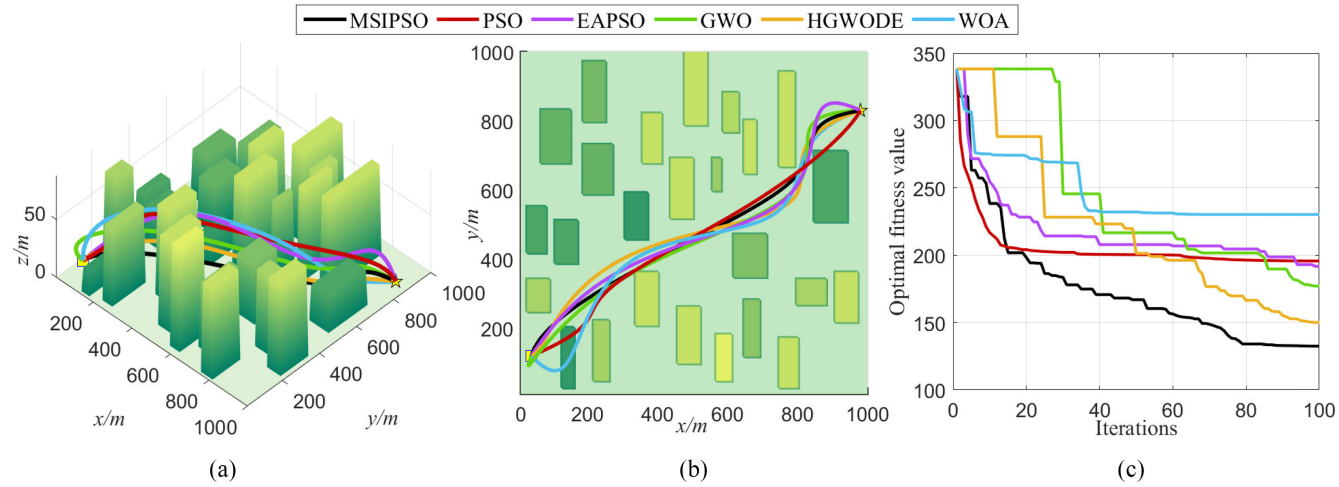

Fig. 11. Comparison of six algorithms in dense environments (scenario 2). (a) 3-D path. (b) Top view. (c) Optimal fitness with iteration.

average fitness of 137.723 and the second lowest convergent iteration of 24 (not significantly different from the minimum value). When $w = [0.6, 0.9]$, the search step size of the particles increases, the convergence speed is the fastest, the average convergent iteration is minimized, but the convergence accuracy decreases and the stability is poor. When $w = [0.2, 0.6]$, the algorithm tends to focus on local search, which slows down the convergence speed. Due to insufficient spatial exploration in the early stages of algorithm iteration, its convergence accuracy is also not good enough. When $w_{\max}$ is too large, particles are prone to excessive oscillation, making it difficult to enter the optimization stage. When $w_{\min}$ is too small, particles tend to stop moving prematurely. Overall, this will result in the algorithm being extremely unstable in terms of convergence speed and accuracy, as shown by $w = [0.1, 0.99]$.

Subsequently, a comparative experiment was conducted on the sensitivity of learning factor parameters under the condition of $w_{\max} = 0.9$ and $w_{\min} = 0.4$ [38]. Usually, to avoid particles biased toward social cognition or individual cognition, let $c_{1_{\min}} = c_{2_{\min}} = c_{\min}$, $c_{1_{\max}} = c_{2_{\max}} = c_{\max}$ [38]. The specific parameter settings and corresponding performances are shown in Fig. 13 and Table V.

TABLE IV
COMPARISON OF PERFORMANCE UNDER DIFFERENT w CONDITIONS

Parameter	Maximum	Minimum	Average	Median	iter.
$w = [0.4, 0.9]$	**144.756**	128.281	**136.795**	**137.273**	24
$w = [0.1, 0.99]$	181.278	**122.424**	151.176	147.757	32
$w = [0.2, 0.6]$	157.763	140.099	148.899	149.148	53
$w = [0.6, 0.9]$	164.157	137.576	149.485	149.209	**20**

It can be seen from this that the optimization effect of the algorithm is best when $c_{\min} = 0.5$, $c_{\max} = 2.5$ with the lowest average fitness of 136.885 and the second lowest convergent iteration of 32. When $c_{\min} = 0.9$, $c_{\max} = 3$, due to the large learning factor, particles will quickly move toward the global and individual optimal points, and the algorithm can converge quickly. But it is easy to ignore other potential solutions and fall into premature convergence, leading to low convergence accuracy. Therefore, its average convergent iteration is the smallest at the 26th generation, but its convergence value is the largest at 165.302. When $c_{\min} = 0.3$, $c_{\max} = 1.5$, the reduced parameter values lead to smaller particle movement steps, resulting in slower convergence. While this configuration

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY GUWAHATI. Downloaded on January 27,2026 at 18:16:58 UTC from IEEE Xplore. Restrictions apply.

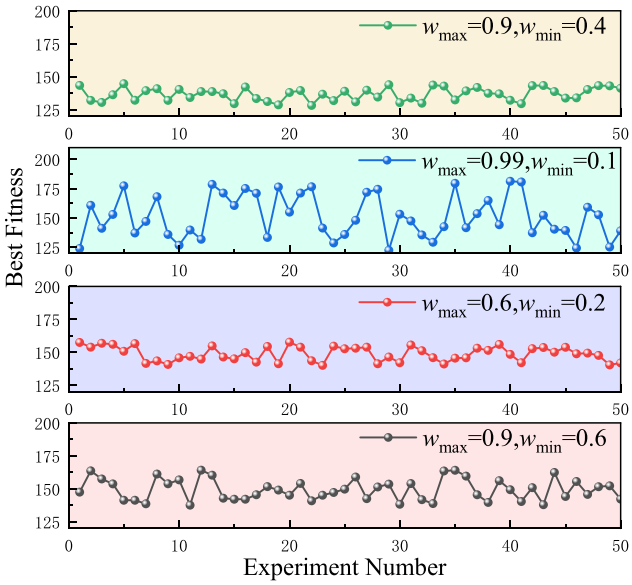

Fig. 12. Comparison of optimal fitness under different w conditions.

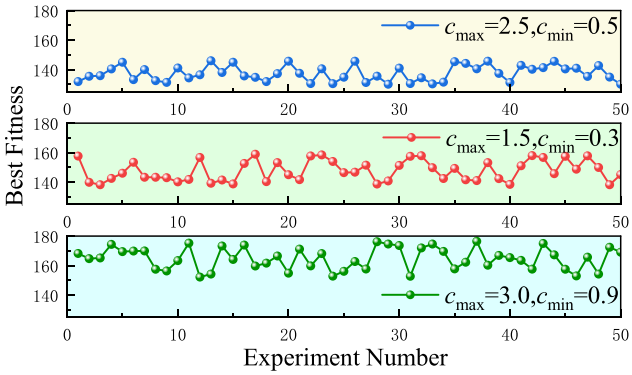

Fig. 13. Comparison of optimal fitness under different c conditions.

TABLE V
COMPARISON OF PERFORMANCE UNDER DIFFERENT c CONDITIONS

Parameter	Maximum	Minimum	Average	Median	iter.
$c = [0.5, 2.5]$	**145.954**	**130.035**	**137.487**	**136.855**	32
$c = [0.3, 1.5]$	158.929	138.279	147.742	146.255	53
$c = [0.9, 3]$	176.466	152.247	164.821	165.302	**26**

enhances the algorithm's exploration capability, it simultaneously weakens the particles' ability to effectively follow both global and individual optima, ultimately compromising convergence accuracy.

From the parameter sensitivity experiment, it can be seen that for MSIPSO, parameter adjustment for different scenarios is still one of the important conditions for achieving optimal algorithm performance. However, overall, the convergence fitness under various parameters remains below 200, demonstrating that MSIPSO itself has relative robustness.

On the whole, the simulation results demonstrate that the proposed methodology outperforms other algorithms in terms of speed, accuracy and stability. Moreover, from simple environment to dense environment, the leadership in these aspects has a trend of gradually expanding. It shows that the proposed methodology has the potential of application in a variety of environments.

VI. CONCLUSION

This study proposes an MSIPSO algorithm for UAV path planning in low altitude urban environments. In MSIPSO, a local deadlock escape strategy was designed to improve the algorithm's ability to find feasible paths in dense environments by lifting velocity restrictions and improving velocity update equations. In addition, to improve the stability of the algorithm, adaptive nonlinear inertia coefficients and asynchronous learning factors were introduced to reduce the impact of parameter selection on the algorithm. Finally, to improve the search efficiency of particles, the algorithm integrates multiple population differentiation evolution strategies, assigning different positions to particles of different qualities to update the equation. Comparative experiments under different simulation environments show that MSIPSO outperforms PSO, GWO, WOA, EAPSO, and HGWODE in terms of stability and convergence performance. Therefore, it is expected to become an effective alternative to other optimizers in low altitude dense obstacle environments, capable of efficiently generating high-quality drone paths.

In the future, we will be committed to deploying MSIPSO in real urban environments, including integrating advanced local planning algorithms to address dynamic obstacles, collaborative optimization with control algorithms, triggering and processing mechanisms in emergency situations, real-time high-precision information collection, and lightweight information transmission between IoT systems and UAV. After completing these works, hardware-in-the-loop (HIL) validation and physical UAV flight testing will be combined to validate the algorithm in physical environments.

REFERENCES

[1] Z. Wang et al., "Efficient autonomous UAV exploration framework with limited FOV sensors for IoT applications," *IEEE Internet Things J.*, vol. 12, no. 1, pp. 713–725, Jan. 2025.
[2] J. Luo, Z. Wang, M. Xia, L. Wu, Y. Tian, and Y. Chen, "Path planning for UAV communication networks: Related technologies, solutions, and opportunities," *ACM Comput. Surv.*, vol. 55, no. 9, pp. 1–37, 2023.
[3] A. V. Savkin and H. Huang, "Multi-UAV navigation for optimized video surveillance of ground vehicles on uneven terrains," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 9, pp. 10238–10242, Sep. 2023.
[4] N. Cheng et al., "AI for UAV-assisted IoT applications: A comprehensive review," *IEEE Internet Things J.*, vol. 10, no. 16, pp. 14438–14461, Aug. 2023.
[5] Y. Jiang, X.-X. Xu, M.-Y. Zheng, and Z.-H. Zhan, "Evolutionary computation for unmanned aerial vehicle path planning: A survey," *Artif. Intell. Rev.*, vol. 57, no. 10, p. 267, 2024.
[6] Y.-C. Wang and C.-Y. Chen, "Efficient path planning for UAVs to recognize chimneys with excessive exhaust emissions," *IEEE Internet Things J.*, vol. 11, no. 10, pp. 18580–18592, May 2024.
[7] Y. Tang, K. Huang, Z. Tan, M. Fang, and H. Huang, "Multi-subswarm cooperative particle swarm optimization algorithm and its application," *Inf. Sci.*, vol. 677, Aug. 2024, Art. no. 120887.
[8] Y. Wu, H. Yin, X. Chen, and M. Zhang, "Multi-circuit route planning for UAVs performing the terrain coverage task," *IEEE Internet Things J.*, vol. 11, no. 13, pp. 23765–23779, Jul. 2024.
[9] N. Bashir, S. Boudjit, and G. Dauphin, "A connectivity aware path planning for a fleet of UAVs in an urban environment: Supplementary material," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 10, pp. 10537–10552, Oct. 2023.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY GUWAHATI. Downloaded on January 27,2026 at 18:16:58 UTC from IEEE Xplore. Restrictions apply.

[10] P. Wan, S. Wang, G. Xu, Y. Long, and R. Hu, "Hybrid heuristic-based multi-UAV route planning for time-dependent data collection," *IEEE Internet Things J.*, vol. 11, no. 13, pp. 24134–24147, Jul. 2024.

[11] D. Kudathanthirige, H. Inaltekin, S. V. Hanly, and I. B. Collings, "Optimum UAV trajectory design for data harvesting from distributed nodes," *IEEE Trans. Commun.*, vol. 72, no. 1, pp. 302–316, Jan. 2024.

[12] J. Li, Y. Xiong, and J. She, "UAV path planning for target coverage task in dynamic environment," *IEEE Internet Things J.*, vol. 10, no. 20, pp. 17734–17745, Oct. 2023.

[13] X. Xu, C. Xie, Z. Luo, C. Zhang, and T. Zhang, "A multi-objective evolutionary algorithm based on dimension exploration and discrepancy evolution for UAV path planning problem," *Inf. Sci.*, vol. 657, Feb. 2024, Art. no. 119977.

[14] A. Beishenalieva and S.-J. Yoo, "Multiobjective 3-D UAV movement planning in wireless sensor networks using bioinspired swarm intelligence," *IEEE Internet Things J.*, vol. 10, no. 9, pp. 8096–8110, May 2023.

[15] J. Wang, Y. Li, R. Li, H. Chen, and K. Chu, "Trajectory planning for UAV navigation in dynamic environments with matrix alignment Dijkstra," *Soft Comput.*, vol. 26, no. 22, pp. 12599–12610, 2022.

[16] Z. Zhang, J. Wu, J. Dai, and C. He, "Optimal path planning with modified a-star algorithm for stealth unmanned aerial vehicles in 3D network radar environment," *Proc. Inst. Mech. Eng., Part G, J. Aerosp. Eng.*, vol. 236, no. 1, pp. 72–81, 2022.

[17] Z. Qadir, F. Ullah, H. S. Munawar, and F. Al-Turjman, "Addressing disasters in smart cities through UAVs path planning and 5G communications: A systematic review," *Comput. Commun.*, vol. 168, pp. 114–135, Feb. 2021.

[18] A. V. Savkin, C. Huang, and W. Ni, "Joint multi-UAV path planning and LoS communication for mobile-edge computing in IoT networks with RISs," *IEEE Internet Things J.*, vol. 10, no. 3, pp. 2720–2727, Feb. 2023.

[19] J. Chang, N. Dong, D. Li, W. H. Ip, and K. L. Yung, "Skeleton extraction and greedy-algorithm-based path planning and its application in UAV trajectory tracking," *IEEE Trans. Aerosp. Electron. Syst.*, vol. 58, no. 6, pp. 4953–4964, Dec. 2022.

[20] L. Chen, Y. Shan, W. Tian, B. Li, and D. Cao, "A fast and efficient double-tree RRT $*$-like sampling-based planner applying on mobile robotic systems," *IEEE/ASME Trans. Mechatronics*, vol. 23, no. 6, pp. 2568–2578, Dec. 2018.

[21] J. Zhang, Y. Cui, and J. Ren, "Dynamic mission planning algorithm for UAV formation in battlefield environment," *IEEE Trans. Aerosp. Electron. Syst.*, vol. 59, no. 4, pp. 3750–3765, Aug. 2023.

[22] J. Yang, H. Zhang, and P. Ning, "Path planning and trajectory optimization based on improved APF and multi-target," *IEEE Access*, vol. 11, pp. 139121–139132, 2023.

[23] B. López, J. Muñoz, F. Quevedo, C. A. Monje, S. Garrido, and L. Moreno, "4D trajectory planning based on fast marching square for UAV teams," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 6, pp. 5703–5717, Jun. 2024.

[24] D. Dissanayaka, T. R. Wanasinghe, O. De Silva, A. Jayasiri, and G. K. Mann, "Review of navigation methods for UAV-based parcel delivery," *IEEE Trans. Autom. Sci. Eng.*, vol. 21, no. 1, pp. 1068–1082, Jan. 2023.

[25] Y. Wu, T. Liang, J. Gou, C. Tao, and H. Wang, "Heterogeneous mission planning for multiple UAV formations via metaheuristic algorithms," *IEEE Trans. Aerosp. Electron. Syst.*, vol. 59, no. 4, pp. 3924–3940, Aug. 2023.

[26] Y. Shao, L. Zhu, C. Su, and J. Wang, "An improved whale optimization algorithm for multi-robot path planning," *Eng. Optim.*, vol. 57, no. 6, pp. 1–27, 2024.

[27] H. Liu, G. Wu, L. Zhou, W. Pedrycz, and P. N. Suganthan, "Tangent-based path planning for UAV in a 3-D low altitude urban environment," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 11, pp. 12062–12077, Nov. 2023.

[28] X. Yu, N. Jiang, X. Wang, and M. Li, "A hybrid algorithm based on grey wolf optimizer and differential evolution for UAV path planning," *Expert Syst. Appl.*, vol. 215, Apr. 2023, Art. no. 119327.

[29] H. Wang, S. Song, Q. Guo, D. Xu, X. Zhang, and P. Wang, "Cooperative motion planning for persistent 3D visual coverage with multiple quadrotor UAVs," *IEEE Trans. Autom. Sci. Eng.*, vol. 21, no. 3, pp. 3374–3383, Jul. 2024.

[30] J. Cho, S. Ki, and H. Lee, "Predictive path planning of multiple UAVs for effective network hotspot coverage," *IEEE Trans. Veh. Technol.*, vol. 72, no. 12, pp. 16683–16700, Dec. 2023.

[31] X. Chen, Q. Li, R. Li, X. Cai, J. Wei, and H. Zhao, "UAV network path planning and optimization using a vehicle routing model," *Remote Sens.*, vol. 15, no. 9, p. 2227, 2023.

[32] Z. Ma and J. Chen, "Adaptive path planning method for UAVs in complex environments," *Int. J. Appl. Earth Observ. Geoinf.*, vol. 115, Dec. 2022, Art. no. 103133.

[33] Y. Zhang, "Elite archives-driven particle swarm optimization for large scale numerical optimization and its engineering applications," *Swarm Evol. Comput.*, vol. 76, Feb. 2023, Art. no. 101212.

[34] L. Xu, X. Cao, W. Du, and Y. Li, "Cooperative path planning optimization for multiple UAVs with communication constraints," *Knowl.-Based Syst.*, vol. 260, Jan. 2023, Art. no. 110164.

[35] L. Van Nguyen, N. M. Kwok, and Q. P. Ha, "Fermat-weber location particle swarm optimization for cooperative path planning of unmanned aerial vehicles," *Appl. Soft Comput.*, vol. 167, Dec. 2024, Art. no. 112269.

[36] P. S. Lahari and V. Janamala, "Hunter–prey optimization algorithm: A review," *J. Electr. Syst. Inf. Technol.*, vol. 11, no. 1, p. 19, 2024.

[37] I. C. Trelea, "The particle swarm optimization algorithm: Convergence analysis and parameter selection," *Inf. Process. Lett.*, vol. 85, no. 6, pp. 317–325, 2003.

[38] E. H. Houssein, A. G. Gad, K. Hussain, and P. N. Suganthan, "Major advances in particle swarm optimization: Theory, analysis, and application," *Swarm Evol. Comput.*, vol. 63, Jun. 2021, Art. no. 100868.

Fazhan Tao (Member, IEEE) received the Ph.D. degree in electronic and information engineering from Tokyo University of Agriculture and Technology, Tokyo, Japan, in 2017.

He is currently a Lecturer with Henan University of Science and Technology, Luoyang, China. His research interests include deep learning, nonlinear control, robust stability, and energy management of fuel cell/battery hybrid electric vehicles.

Zezheng Chen received the bachelor's degree in automation from Henan University of Technology, Zhengzhou, China, in 2022. He is currently pursuing the master's degree in control engineering with Henan University of Science and Technology, Luoyang, China.

His research interests include unmanned aerial vehicles, path planning, and collaborative control of multiagent systems.

Zhikai Wang received the Ph.D. degree in navigation, guidance and control from Northwestern Polytechnical University, Xian, China, in 2023.

He is currently a Lecturer with the School of Information Engineering, Henan University of Science and Technology, Luoyang, China. His current research interests include aircraft guidance and control, aircraft trajectory planning, multiagent cooperative control.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY GUWAHATI. Downloaded on January 27,2026 at 18:16:58 UTC from IEEE Xplore. Restrictions apply.

Longlong Zhu received the Ph.D. degree with the School of Information Engineering from Henan Univesity Science and Technology, Luoyang, China, in 2024.

He is currently a Lecturer with Henan University of Science and Technology. His research interests include energy management strategy, adaptive cruise control, multiobjective optimization, and hybrid electrical vehicle modeling.

Jun Wang received the Ph.D. degree in agricultural engineering from China Agricultural University, Beijing, China, in 2012.

He is currently a Professor with the School of Information Engineering, Henan University of Science and Technology, Luoyang, China. His research interests include wireless sensor networks, collaborative sensing, data fusion, and sensor stability.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY GUWAHATI. Downloaded on January 27,2026 at 18:16:58 UTC from IEEE Xplore. Restrictions apply.